시계열분석

2주차: 시계열 데이터 탐색1

시계열 자료(Time series data)란?

시간 순서에 따라 관측된 데이터

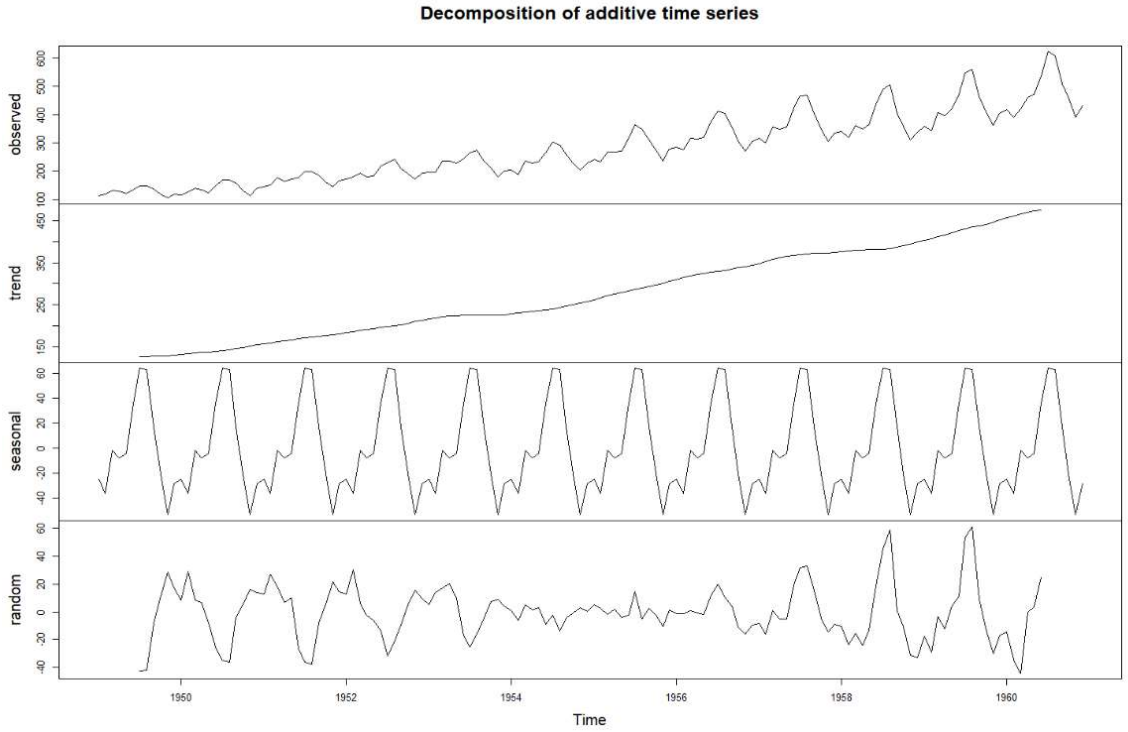
- 시간 의존성 및 순서 정보 존재
- 자기상관성, 계절성, 추세 등 고유한 패턴 존재

예) **A**제품의 **1**기부터 **t**기까지 판매량 Y_i와 그 상품의 품질 X_i

$$\begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_t \end{bmatrix} = \beta_0 + \beta_1 \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_t \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \dots \\ \varepsilon_t \end{bmatrix}$$

시계열 데이터의 주요 구성 요소

- 추세**(Trend):** 장기적인 증가/감소 추세
- 계절성**(Seasonality):** 정기적, 주기적 변화
- 불규칙성**(Irregular/Residual):** 예측 불가능한 변동



R에서 시계열 데이터 다루기

기본 함수:

- ts(): 시계열 객체 생성
- plot.ts(): 시계열 데이터 시각화

실습1) R에서 난수 생성(rnorm)을 이용해 100개의 데이터를 만들자. 이를 ts() 함수를 사용해 시계열 객체(주기: 12, 시작: 2020년 1월)로 변환해보자. plot() 함수를 이용해 그래프로 시각화해보자.

```
set.seed(123)
data <- rnorm(100)
ts_data <- ts(data, frequency = 12, start = c(2020, 1))
plot(ts_data, main = "랜덤 시계열 데이터", ylab = "값", xlab = "시간")
```

R에서 시계열 데이터 다루기

- as.POSIXct() 함수: 날짜와 시간을 다루기 위해 사용되는 함수로, 주어진 문자형(또는 다른 형식의) 날짜 데이터를 POSIXct 객체로 변환해줌
- 이 변환을 통해 날짜 및 시간 데이터를 내부적으로 초 단위의 숫자(1970–01–01 00:00:00 UTC 기준)로 저장할 수 있으며, 이를 활용해 시간 연산, 비교, 정렬, 시계열 분석 등의 작업을 보다 효율적으로 수행할 수 있음

```
datetime <- as.POSIXct("2025-02-27 15:30:00", format = "%Y-%m-%d %H:%M:%S", tz = "UTC")
```

  - tz 인수: 원하는 시간대를 지정(시간대는 OlsonNames() 함수 참고)

시계열 데이터 탐색 기법

기본 탐색 방법:

- 시각화 (line plot, seasonal plot 등)
- 이동평균(Moving Average)
- 시계열 분해(decomposition)

실습2) R 내장 데이터셋인 AirPassengers를 불러와보자. decompose() 함수를 사용해 시계열 데이터를 추세, 계절성, 잔차로 분해하고 분해된 결과를 각각 시각화해보자.

```
data("AirPassengers")
decomp <- decompose(AirPassengers)
plot(decomp)
```

패키지 소개 – zoo

- 정규 및 불규칙 시계열 데이터를 효율적으로 다루기 위한 패키지
- 데이터와 인덱스를 분리하여 관리하므로 다양한 시계열 분석 및 조작에 용이함

zoo(x, order.by = dates, ...): 시계열 객체 생성

x: 시계열 데이터 (벡터, 행렬, 데이터프레임 등)

order.by: 데이터에 대응하는 인덱스(날짜, 시간, 숫자 등); 데이터 길이와 일치해야 함

```
library(zoo)
data <- rnorm(10)                    # 임의의 데이터 생성
dates <- as.Date("2025-01-01") + 0:9     # 날짜 인덱스 생성
z <- zoo(data, order.by = dates)        # zoo 객체 생성
plot(z, main = "zoo 객체 시각화", xlab = "날짜", ylab = "값")
```

패키지 소개 – zoo

as.zoo(): 다른 타입의 객체(예: ts 객체)를 zoo 객체로 변환

```
ts_data <- ts(rnorm(10))
zoo_data <- as.zoo(ts_data)
```

index(): zoo 객체의 인덱스(시간 정보)를 추출

```
time_index <- index(z)
print(time_index)
```

coredata(): zoo 객체의 데이터 부분만 추출

```
values <- coredata(z)
print(values)
```

패키지 소개 – zoo

merge.zoo(): 여러 zoo 객체를 병합하여 하나의 시계열 객체로 생성

```
z1 <- zoo(rnorm(10), order.by = dates)
z2 <- zoo(rnorm(10), order.by = dates)
merged <- merge(z1, z2)
plot(merged, main = "병합된 zoo 객체")
```

lag.zoo(): 시계열 데이터를 지연(lag) 처리하여 이전 시점 데이터를 참조

```
lag_z <- lag(z, k = 1)   # 1단위 시차
plot(lag_z, main = "1단위 지연된 시계열")
```

패키지 소개 – data.table

- 롤링 조인 (Rolling Join): 두 데이터 테이블을 시간 기준으로 조인할 때, 정확한 일치가 없으면 가장 가까운(주로 과거) 값을 자동으로 할당사용
- 예: 시계열 데이터에서 특정 시점의 값이 없는 경우, 이전 시점의 값을 채워 넣을 때 유용

```
library(data.table)
# 예시 데이터 생성
dt1 <- data.table(
  time = as.POSIXct(c("2025-01-01 10:00:00", "2025-01-01 10:05:00")),
  value1 = c(10, 20))
dt2 <- data.table(
  time = as.POSIXct(c("2025-01-01 10:02:00", "2025-01-01 10:07:00")),
  value2 = c(30, 40))
```

패키지 소개 – data.table

키설정:

setkey(data, 변수)

롤링 조인: dt2의 각 시각에 대해 dt1의 가장 가까운 과거 값을 찾음

```
result <- dt1[dt2, roll = TRUE]
print(result)
```

패키지 소개 – data.table

- := (동적 할당 연산자): 기존 컬럼 수정 또는 새 컬럼 추가 시 메모리를 복사하지 않고 바로 업데이트

```
DT <- data.table(x = 1:5, y = rnorm(5))
# 새 컬럼 z를 추가하여 x와 y의 합을 계산
DT[, z := x + y]
print(DT)
```

- .() (리스트 축약 표기): list() 대신 사용 가능하며, 여러 값을 반환하거나 컬럼 선택 시 간결하게 표현

```
# 각 컬럼의 평균과 합계를 계산하여 반환
summary <- DT[, .(mean_y = mean(y), sum_x = sum(x))]
print(summary)
```

패키지 소개 – data.table

- .N (행 개수)용도: 그룹별 또는 전체 행의 개수를 쉽게 계산하는 내장 변수

```
# 그룹별(예: x의 값에 따른) 행 개수 계산
count_by_x <- DT[, .N, by = x]
print(count_by_x)
```

2주차 과제

- eclass에 있는 2주차 과제를 푸세요.
- 문제 #10은 아래 데이터를 참고하세요.

온도 데이터(dt1)

time	temperature
2025-01-01 00:00:00	10
2025-01-01 01:00:00	12
2025-01-01 02:00:00	11
2025-01-01 03:00:00	13

습도 데이터(dt2)

time	humidity
2025-01-01 00:30:00	70
2025-01-01 01:45:00	65
2025-01-01 02:15:00	68

누락된 데이터 다루기

- 대치법(imputation): 데이터셋 전체의 관측에 기반하여 누락된 데이터를 채워 넣는 방법

- 보간법(interpolation): 대치법의 한 형태로 인접한 데이터를 사용하여 누락된 데이터를 추정하는 방법

- 영향받은 기간 삭제: 누락된 데이터의 기간을 완전히 사용하지 않는 방법

누락된 데이터 다루기

"Lec2_UNRATE.csv" 데이터는 1948년부터 2018년까지 미국 정부가 발표한 월간 실업 자료이다. "Lec2_sample.txt" 데이터는 임의로 된 행 번호이다. "Lec2_sample_high.txt" 데이터는 실업률이 높은 데이터의 임의의 행번호이다. "Lec2_UNRATE.csv" 데이터를 불러와 누락될 데이터의 아래 코드를 활용하여 데이터를 불러와보자.

```
unemp <- fread("Lec2_UNRATE.csv")
unemp[, DATE := as.Date(DATE)]      #DATE 열을 Date 타입으로 변환
setkey(unemp, DATE)
missing_sample <- scan("Lec2_sample.txt", what = integer())    #정수(integer) 형태의 벡터로 저장
unemp_missing <- unemp[-missing_sample]
missing_sample_high <- scan("Lec2_sample_high.txt", what = integer())
unemp_missing_high <- unemp[-missing_sample_high]
```

누락된 데이터 다루기

누락된 데이터를 처리하기 위해 전체 월별 날짜 범위를 생성한 뒤, 기존 데이터를 해당 날짜 범위와 조인하여 결측치 여부를 확인해보자.

```
all_dates <- seq(from = unemp$DATE[1], to = tail(unemp$DATE, 1), by = "months")


unemp_missing = unemp_missing[J(all_dates), roll=0]
#all_dates를 키로 하는 조인을 수행하는데 정확히 일치하는 날짜만 매칭하고, 없는 날짜는 결측(NA)으로 처리


unemp_missing[, rpt := is.na(UNRATE)]    #UNRATE가 결측치인지 여부를 나타내는 새로운 컬럼(rpt)을 추가
```

마찬가지로 unemp_missing_high에 대해서도 조인을 수행하고 없는 날짜는 NA로 처리 후, 결측치 여부를 나타내는 새로운 컬럼 rpt 를 추가해보자.

누락된 데이터 다루기 – Forward Fill

- 포워드 필(forward fill)은 누락된 값이 나타나기 직전의 값으로 누락된 값을 채우는 가장 간단한 방법.

  na.locf(data, na.rm=F): 결측치를 이전 관측값으로 대치

zoo 패키지 필요

앞의 unemp_missing 과 unemp_missing_high 데이터에 impute_ff 열을 생성하여 포워드 필 된 값을 넣어보자.

```
unemp_missing[, impute_ff := na.locf(UNRATE, na.rm = FALSE)]
unemp_missing_high[, impute_ff := na.locf(UNRATE, na.rm = FALSE)]
```

누락된 데이터 다루기 – Forward Fill

```
narm = F
-> 시작이 NA이면 채울 수 없음
narm = T
-> 처음이 NA여도 값을 가져와서 채움
```

원본 데이터인 unemp의 300부터 400번까지의 값을 그래프로 표현해보자. 이 때, x 축을 DATE, y 축을 UNRATE, 색은 검정색, 그래프 타입은 점선 형태로 그려보자.

```
unemp[300:400, plot(DATE, UNRATE, col = 1, type = 'b')]
```

여기에 unemp_missing 데이터를 선 형태로 추가해보자. 이 때, x 축을 DATE, y 축을 impute_ff, 색은 빨간색으로 그려보자.

```
unemp_missing[300:400, lines(DATE, impute_ff, col = 2)]
```

포워드 필 된 값을 점으로 표현해보자. 색은 빨간색, 기호는 24번으로 그려보자.

```
unemp_missing[300:400][rpt == TRUE, points(DATE, impute_ff, col = 2, pch=24)]
```

누락된 데이터 다루기 – Forward Fill

원본 데이터인 unemp의 300부터 400번까지의 값을 그래프로 표현해보자. 이 때, x 축을 DATE, y 축을 UNRATE, 색은 검정색, 그래프 타입은 점선 형태로 그려보자.

여기에 unemp_missing_high 데이터를 선 형태로 추가해보자. 이 때, x 축을 DATE, y 축을 impute_ff, 색은 초록색으로 그려보자.

포워드 필 된 값을 점으로 표현해보자. 색은 초록색, 기호는 19번으로 그려보자.